

Code review — academy-hub-api

Runde de review pe proiectul tău, cea mai nouă sus.

Runda 4 — 2026-09-25 — sha 8ecffc9

Scop: CRUD-ul complet pe cele patru module + starea celor 6  din runda 3.

Proiectul **compilează** (`mvn compile` ). **Nu pornește** pe o bază de date nouă — vezi B1; restul probelor le-am făcut pe o schemă creată înainte, exact ca a ta. Toate constatările au fost reproduse prin rulare: copie în afara repo-ului, H2 în loc de MySQL, portul 8094, **peste 30 de cereri HTTP reale** plus două porniri succesive ale aplicației. Codul din repo nu a fost modificat.

Ce s-a închis

- **CRUD complet pe toate patru modulele** (runda 3, B6) — `POST`, `PUT`, `DELETE` există acum pe student, course, book și enrollment. Cele trei command service-uri care stăteau nefolosite sunt legate la HTTP.
- **createBook funcționează** (runda 3, B5 · runda 2, B4 — deschis de două runde) — `BookCreateRequest` poartă acum `studentId`, încarci studentul, `student.addBook(newbook)`. Măsurat: `POST /api/books` → **200**, cartea intră în bază. Exact fixul cerut.
- **@NotNull pe EnrollmentUpdateRequest și EnrollmentDeleteRequest** (runda 3, B4) — `PUT /api/enrollments/{id}` cu body `{}` răspunde acum **400**, nu 500.
- **Enrollment.equals nu mai crapă** (runda 3, M6 · runda 2, B6) — compară `id`-ul, nu `student.getId()`. Închis după două runde.
- **deleteStudent primește id, nu email** (runda 3, M7 · runda 2, B7) — semnătura s-a schimbat pe `UUID`, iar `StudentDeleteRequest` nu mai e folosit la ștergere.
- **findById / getById pe curs nu mai fac două interogări** (runda 3, M2 parțial) — `findById` e acum `orElseThrow(NoCourseFound::new)`. Le-ai reparat în `CourseQueryService`; în `StudentQueryService` tiparul a rămas (M3 mai jos).
- **Constrângerea UNIQUE (student_id, course_id)** cerută la B3 — pusă pe `@Table` . Din păcate, împreună cu ea au intrat două constrângeri care nu trebuiau — B1.

Critice

B1 — Aplicația nu mai pornește pe o bază de date nouă

`enrollment/models/Enrollment.java:26` și `:31`

```
@JoinColumn(name = "student_id", unique = true, nullable = false)
Student student;

@JoinColumn(name = "course_id", unique = true, nullable = false)
Course course;
```

CUM E ACUM: pornit pe o bază goală, aplicația **cade la pornire**, în `seed()` :

```
APPLICATION FAILED TO START
Caused by: JdbcSQLIntegrityConstraintViolationException:
Unique index or primary key violation:
"PUBLIC.UK81Q4XON4LWV13AY3G8FXHJ7F INDEX ON PUBLIC.ENROLLMENT(STUDENT_ID)"
SQL statement: insert into enrollment (course_id,created_at,student_id,id)
```

Am izolat cauza: am șters **doar** cele două `unique = true`, nimic altceva, și aplicația a pornit în 2,4 secunde, `GET /api/students` → 200.

Iar pe baza ta pornește. Am verificat și asta: am creat schema o dată cu codul fără `unique = true`, apoi am pornit peste ea codul `cu unique = true` — a pornit fără o vorbă. În log, singurele `alter table` de pornire sunt:

```
alter table if exists enrollment
add constraint uk_enrollment_student_course unique (student_id, course_id)
```

DE CE: `unique = true` pe un `@JoinColumn` nu spune „perechea să fie unică” — spune „coloana asta să fie unică”. Pusă pe amândouă coloanele separat, cere două lucruri absurde pentru o tabelă de legătură:

- `student_id` unic → **un student poate avea o singură înscriere în toată baza**
- `course_id` unic → **un curs poate avea un singur student înscris**

Seed-ul tău înscrie primul student la trei cursuri. A doua linie din `insert` lovește indexul unic pe `student_id` și aplicația se oprește. Constrângerea corectă, cea pe pereche, o pusese deja `@Table(uniqueConstraints = ...)` — cele două `unique = true` nu o întăresc, o contrazic.

Partea care explică de ce n-ai văzut nimic: `ddl-auto: update` **compară doar structura pe care o poate numi**. Constrângerile cu nume din `@Table` le caută și le adaugă (`uk_enrollment_student_course` a apărut în log). Indexul unic generat implicit de `unique = true` are un nume inventat de Hibernate (`UK81Q4X...`), nu-l caută și nu-l adaugă pe o tabelă care există deja. Deci pe MySQL-ul tău, cu tabela veche, cele două linii sunt literă moartă. Pe orice bază nouă — laptopul unui coleg, un container proaspăt, CI — sunt primul lucru care se execută, și aplicația nu mai pornește deloc.

E aceeași formă de capcană ca la coloanele pe care `ddl-auto: update` nu le modifică niciodată: codul și baza ta au divergat, iar baza ta e singura pe care bug-ul nu se vede.

FIX: scoți `unique = true` din amândouă `@JoinColumn`-urile. `nullable = false` rămâne — acela e corect. Unicitatea perechii o ține deja `@Table`.

Cum verifici că ai reparat: ștergi baza (`DROP DATABASE academy_hub_db`), pornești aplicația. Dacă pornește și `GET /api/enrollments` întoarce înscrierile din seed, e închis. Atât timp cât n-o ștergi, testul nu-ți spune nimic.

B2 — Fiecare pornire a aplicației șterge tot ce a scris API-ul

Runner.java:80-95

```
@Override
@Transactional
public void run(String... args) throws Exception {
    reset();
    seed();
    clearPersistence();
}

void reset(){
    bookRepository.deleteAllInBatch();
    enrollmentRepository.deleteAllInBatch();
    courseRepository.deleteAllInBatch();
    studentRepository.deleteAllInBatch();
}
```

CUM E ACUM: reprodus în doi pași, pe aceeași bază, fără să ating nimic între ei —

```
11 studenți în bază (8 din seed + 3 adăugați de mine prin POST /api/students)
    -> repornesc aplicația
8 studenți în bază
```

Cei trei pe care îi crease API-ul — `ion.pop@test.ro`, `maria2@test.ro`, `gol@test.ro` — nu mai există. Nici cărțile, nici înscrierile lor. `seed()` mai face pe deasupra încă un rând de `deleteAll()` pe toate patru tabelele (`Runner.java:118-121`), deci ștergerea se întâmplă de două ori.

DE CE: un `CommandLineRunner` nu e un script pe care îl rulezi când vrei — e o metodă pe care Spring o cheamă **la fiecare pornire**, automat, pentru că `Runner` e `@Component` și implementează interfața. Cât timp era tot comentat (runda 3) nu conta. Acum că ai decommentat `reset()` și `seed()`, fiecare repornire — un restart din IntelliJ, un deploy, o cădere cu restart automat — trece un `DELETE FROM` peste toate cele patru tabelele.

Până la runda asta nu se vedea, pentru că datele intrau tot din `Runner`: ștergea și rescria exact ce ștersese. De când ai API, datele au **două surse** — seed-ul și clienții HTTP — și numai una dintre ele supraviețuiește pornirii. Ăsta e momentul în care un seeder încetează să fie un ajutor și devine o pierdere de date.

FIX: seed-ul nu trebuie să ruleze la fiecare pornire și mai ales nu pe baza pe care lucrezi. Îl pui pe un profil separat, `@Profile("seed")`, și îl pornești explicit doar când vrei date de test. Varianta minimă, dacă vrei să mergi mai repede: comentezi `reset()` și `seed()` înapoi, exact cum erau la runda 3.

B3 — Tot niciun `@RestControllerAdvice`: 12 situații diferite, toate 500 (a 4-a rundă)

Căutat în tot `src/`: zero `@RestControllerAdvice`, zero `@ExceptionHandler`, zero `@ResponseStatus`. Cerut la runda 1, la runda 2 și la runda 3.

CUM E ACUM: la runda 3 erau șase situații; acum, cu CRUD-ul expus pe toate modulele, sunt douăsprezece. Toate reproduse azi:

Cerere	Ce ar trebui	Ce primești
PUT /api/students/{id} cu id inexistent	404	500
DELETE /api/students/{id} cu id inexistent	404	500
PUT /api/students/{id} cu emailul altui student	409	500
POST /api/courses cu nume deja existent	409	500
PUT /api/courses/{id} cu id inexistent	404	500
DELETE /api/courses/{id} cu id inexistent	404	500
PUT /api/courses/{id} cu numele altui curs	409	500
POST /api/books cu studentId inexistent	404	500
POST /api/books carte deja atribuită	409	500
PUT /api/books/{id} cu createdAt în viitor	400	500
POST /api/enrollments pereche existentă	409	500
PUT /api/enrollments/{id} cu id inexistent	404	500
GET pe orice listă goală	200 []	500

Corpul, identic la toate:

```
{"timestamp":"...", "status":500, "error":"Internal Server Error",  
  "path":"/api/students/..."}
```

Ai scris zece clase de excepție cu mesaje în `ExceptionConstants`. **Niciunul dintre mesajele acelea nu ajunge vreodată la un client.** Există doar în log-ul tău.

DE CE: mecanismul e cel de la runda 3 și n-a plecat: când excepția iese din controller, `DispatcherServlet` întreabă pe rând fiecare `HandlerExceptionResolver` dacă o recunoaște. `@ExceptionHandler` -ele tale ar fi întrebată primele — nu există. `ResponseStatusExceptionResolver` caută `@ResponseStatus` pe clasa excepției — nu ai.

`DefaultHandlerExceptionResolver` știe doar excepțiile lui Spring. Niciunul nu o revendică, cererea e redirectată intern spre `/error`, iar `BasicErrorController` o tratează ca pe orice eroare necunoscută.

Ce e nou la runda asta e cât de departe ajunge efectul. Uită-te la ultimele două rânduri din tabel: PUT cu `createdAt: "2099-12-31"` dă 500 pentru că `@PastOrPresent` de pe `Book.createdAt` prinde valoarea abia la flush, în Hibernate, după ce cererea a trecut de controller. Iar POST /api/courses cu nume duplicat dă 500 deși ai scris anume

`CourseNameAlreadyInUse` pentru cazul ăsta. Greșeala clientului, defectul tău intern și regula ta de business arată, din afară, absolut identic. Un frontend nu are din ce să distingă „ai trimis o dată greșită” de „serverul e căzut”.

FIX: o singură clasă `@RestControllerAdvice`, cu câte un `@ExceptionHandler` pe familie: not-found → 404, conflict/deja-există → 409, validare → 400. Întoarce un corp cu `message`, luat din `ExceptionConstants` -urile pe care le-ai scris deja.

B4 — Verificarea de duplicat la carte refuză cărți pe care studentul nu le are

`book/services/BookCommandServiceImpl.java:37-40`

```
if(!bookRepository.findByStudentId(bookCreateRequest.studentId()).isEmpty() &&
    bookRepository.existsByName(bookCreateRequest.name())) {
    throw new BookAlreadyAssignedToThisStudent();
}
```

CUM E ACUM: patru cereri, în ordinea în care le-am trimis. Studentul A avea deja `Effective Java`, `Clean Code`, `Spring Start Here`. Studentul B era nou, fără nicio carte.

```
POST /books {Refactoring, A} -> 200 corect
POST /books {Refactoring, A} -> 500 corect, e duplicat real
POST /books {Refactoring, B} -> 200 corect, B n-avea cartea
POST /books {Effective Java, B} -> 500 GREȘIT – B nu are Effective Java
```

Ultimul rând e bug-ul: cartea e la A, B n-o are, iar cererea e refuzată.

DE CE: cele două condiții din `if` nu vorbesc despre același lucru și nimic nu le leagă. Prima întreabă „studentul ăsta are vre-o carte?”. A doua întreabă „numele ăsta există la cineva, oriunde în baza de date?”. Din ele două nu rezultă întrebarea care te interesa: „studentul ăsta are cartea asta?”.

De-aia rezultatul pare aleatoriu: pentru un student fără cărți prima condiție e falsă și verificarea nu se face niciodată, iar pentru un student care are măcar o carte a doua condiție transformă orice titlu deja existent în școală într-un titlu interzis. Efectul practic, pe un catalog adevărat: al doilea student care cere `Clean Code` nu o poate primi, pentru că altcineva o are — dar numai dacă are deja o carte. Un `&&` între două întrebări care nu se referă la aceeași mulțime nu e o verificare mai strictă, e o verificare despre altceva.

FIX: o singură întrebare, despre pereche, cum ai făcut deja la înscrieri: `existsByStudentIdAndName(studentId, name)` în `BookRepository` (Spring Data ți-o generează din nume), și `if` pe rezultatul ei.

B5 — `orElseThrow()` fără argument: 7 locuri în care 404-ul devine 500

`book/services/BookCommandServiceImpl.java:44`

```
Student student = studentRepository.findById(bookCreateRequest.studentId())
    .orElseThrow();
```

CUM E ACUM:

```
POST /api/books {"name":"Ceva","studentId":"00000000-0000-0000-0000-000000000000"}
-> HTTP 500
```

`StudentNotFound` este importat în fișier, la linia 10. Nu e chemat nicăieri.

DE CE: `orElseThrow()` fără argument aruncă `NoSuchElementException` — o excepție tehnică din JDK, care nu spune nimic despre domeniul tău. `orElseThrow(StudentNotFound::new)` aruncă excepția ta, cea pe care un advice o poate mapa pe 404 și al cărei mesaj poate ajunge la client. Diferența nu se vede acum, pentru că fără advice (B3) ambele ies 500 — dar în ziua în care scrii advice-ul, cele cu `orElseThrow()` gol vor rămâne singurele care răspund tot 500, și o să le cauți una câte una.

Importul nefolosit e semnul: știai care e excepția corectă, ai lăsat-o pe cea implicită.

FIX: fiecare `orElseThrow()` primește excepția lui de domeniu.

Aceeași regulă, în toate locurile: `BookCommandServiceImpl.java:44`, `:68` .

`EnrollmentCommandServiceImpl.java:68`, `:82`, `:83`, `:84` . `StudentQueryServiceImpl.java:69` . Șapte locuri — de asta îl numesc bug de regulă, nu de linie.

B6 — Unicitatea nu e verificată la update, nici pe student, nici pe curs

```
student/services/StudentCommandServiceImpl.java:65-74
course/services/CourseCommandServiceImpl.java:50-56
```

CUM E ACUM:

```
PUT /api/students/{id} cu emailul unui alt student -> 500
PUT /api/courses/{id} cu numele unui alt curs -> 500
```

Datele nu se stric — baza refuză scrierea. Dar clientul primește „serverul a picat” pentru o greșeală a lui, iar mesajele `EmailAlreadyUsed` și `CourseNameAlreadyInUse`, pe care le-ai scris exact pentru asta, nu se execută.

DE CE: la creare ai verificat: `addStudent` cheamă `existsByEmail` (`:33`), `createCourse` cheamă `existsByNameIgnoreCase` (`:27`). La update, amândouă metodele încarcă entitatea și trec direct pe `setEmail` / `setName`, fără nicio întrebare. Regula a fost scrisă pentru drumul pe care l-ai testat — crearea — și n-a fost dusă pe al doilea drum care duce în aceeași coloană unică.

Asta e a doua rundă la rând cu aceeași constatare (runda 3, M8) și e tiparul pe care ți l-am semnalat și la company-management: repara locul semnalat, nu regula. Diferența e că acum ambele drumuri sunt deschise prin HTTP, deci oricine poate ajunge pe cel nereparat.

Atenție la forma verificării: la update întrebarea corectă nu e „există emailul?” — e „există emailul la **altcineva?**”. Studentul pe care îl modifici are deja emailul lui, deci `existsByEmail` copiat din `addStudent` ar refuza orice `PUT` care nu schimbă emailul. E exact capcana de la B7.

FIX: `studentRepository.findById(email).filter(s -> !s.getId().equals(id))` → dacă e prezent, `EmailAlreadyUsed`. Analog pentru curs, cu numele.

B7 — PUT pe o înscriere cu aceeași pereche → 500 (runda 3, B3 — NEATINS)

enrollment/services/EnrollmentCommandServiceImpl.java:79

```
if(enrollmentRepository.findByStudentIdAndCourseId(eur.studentId(),
    eur.courseId()).isPresent()) {
    throw new StudentAlreadyEnrolledInThisCourse();
}
```

CUM E ACUM: reprodus din nou, identic cu runda 3 —

```
POST /api/enrollments {S, C}          -> 200, id = 7375ef10
PUT  /api/enrollments/7375ef10 {S, C} -> 500      <- acelasi PUT, a doua oara
PUT  /api/enrollments/7375ef10 {S, C2} -> 200      <- mutat pe alt curs, merge
```

Deci: mutarea pe alt curs merge, retrimiteră aceleiași înscrieri nu. Un client care reîncearcă după un timeout primește „studentul e deja înscris” despre înscrierea pe care tocmai o salva.

DE CE: e explicația de la runda 3, pe care o repet pentru că linia n-a fost atinsă: verificarea întreabă „există vreo înscriere cu perechea asta?”, dar întrebarea corectă la update e „există **altă** înscriere cu perechea asta?”. Rândul pe care îl modifici se găsește pe sine în căutare și se declară propriul duplicat.

Constrângerea **UNIQUE** pe pereche, pe care ai pus-o între timp, nu ajută aici — ea ar fi lăsat trecerea, pentru că **UPDATE** -ul scrie aceleași valori în același rând. Bug-ul e întreg în **if**.

FIX: cel din runda 3, necontestat:

```
enrollmentRepository.findByStudentIdAndCourseId(eur.studentId(), eur.courseId())
    .filter(e -> !e.getId().equals(id))
    .ifPresent(e -> { throw new StudentAlreadyEnrolledInThisCourse(); });
```

B8 — Listele goale răspund tot 500, în 13 locuri (runda 3, B1 — NEATINS)

student/services/StudentQueryServiceImpl.java:29 , :59 , :98 , :105 , :112 , :124 .
course/services/CourseQueryServiceImpl.java:36 , :47 , :65 , :85 .
book/services/BookQueryServiceImpl.java:30 , :37 , :45 .
enrollment/services/EnrollmentQueryServiceImpl.java:29

CUM E ACUM: pe o bază cu date, cele patru **GET** de listă răspund 200 — de asta pare în ordine. Pe o bază goală, toate patru răspund 500. Iar **getAll()** din **StudentQueryService** e chemat de **GET /api/students**, deci e drumul pe care îl parcurge orice client la prima deschidere a aplicației.

DE CE: „nu există niciun student” nu e o eroare, e un răspuns — și răspunsul corect este **[]**. Eroarea înseamnă „cererea ta n-a putut fi onorată”; aici a fost onorată perfect. Un frontend care primește **200 []** afișează „nicio înregistrare”; unul care primește 500 afișează „a picat serverul” și nu are cum să deosebească baza goală de baza căzută.

Singurul caz în care lipsa e într-adevăr eroare e când ceri **un anume** lucru după id — acolo 404 e corect, pentru că resursa pe care ai numit-o nu există. O listă nu numește nimic, deci nu poate lipsi.

Ai reparat între timp `CourseQueryServiceImpl.findById` — `orElseThrow(NoCourseFound::new)`, exact cum trebuie, pentru că **acolo** e vorba de un lucru anume. Diferența dintre cele două cazuri o ai deci înțeleasă; e doar neaplicată pe liste.

FIX: scoți `throw` din cele 13 metode de listare și returnezi lista așa cum vine. `NoStudentsFound`, `NoCourseFound`, `NoBookFound`, `NoEnrollmentsFound` rămân nefolosite și se șterg.

🟡 Importante

M1 — `CourseCommandServiceImpl` și `EnrollmentCommandServiceImpl` nu au `@Transactional` pe nicio metodă (runda 3, M9 · runda 2, M6 — a treia rundă). În `CourseCommandServiceImpl.java:9` adnotarea e **importată** și nepusă pe nimic. `EnrollmentCommandServiceImpl` nu o are nici importată. `BookCommandServiceImpl` o are doar pe `createBook`, nu pe `deletebook` / `updatebook`. Fără tranzacție, „verific apoi scriu” poate fi întrerupt de o a doua cerere între cele două operații — iar acum toate metodele sunt expuse prin HTTP, deci două cereri simultane sunt normalul, nu excepția.

M2 — `findByDepartmentOrderByNameAsc` nu mai sortează. `CourseQueryServiceImpl.java:61`: metoda se numește `...OrderByNameAsc`, dar în corp cheamă `courseRepository.findByDepartment(department)` — derived query-ul cu sortare nu mai e apelat. Numele metodei promite o ordine pe care nu o livrează. (S-a întâmplat la refacerea lui `CourseSummary` din interfață în record.)

M3 — interogarea rulează de două ori, tipar prezent în 12 locuri.

`StudentQueryServiceImpl.java:29,66,74,82,98,105,112` · `BookQueryServiceImpl.java:37,45` · `EnrollmentCommandServiceImpl.java:42+52`, `:45+53`, `:63+66`. Semnalat la runda 2 (M4) și runda 3 (M2); în `CourseQueryServiceImpl` l-ai închis, în celelalte a rămas.

M4 — verificări `== null` care nu se pot declanșa niciodată. `EnrollmentCommandServiceImpl.java:42` și `:45`:

```
if(studentQueryService.getStudentById(...)==null) { throw new StudentIdNotFound(); }
```

`getStudentById` nu întoarce `null` — aruncă excepție dacă nu găsește (`StudentQueryServiceImpl.java:66-69`). Deci `if` -ul e mort: dacă studentul lipsește, excepția a fost aruncată deja de linia dinainte, iar `StudentIdNotFound` nu se execută nicodată. Același lucru la curs. Plus `StudentQueryServiceImpl.java:36,43,50,58` pe liste (runda 2, M5 · runda 3, M3).

M5 — `POST` răspunde 200, nu 201 — pe toate patru modulele acum (`StudentController.java:35` · `CourseController.java:34` · `BookController.java:37` · `EnrollmentController.java:33`). La creare se întoarce `201 Created` cu antetul `Location` spre resursa nouă. Semnalat la runda 3 (M4), când era un singur loc; acum sunt patru.

M6 — `DELETE /api/enrollments` ia id-urile din corpul cererii, nu din cale. `EnrollmentController.java:43-46`. Merge (200, l-am rulat), dar multe proxy-uri și clienți HTTP aruncă corpul unui DELETE. Forma așteptată: `DELETE /api/enrollments/{id}`, plus — dacă vrei și ștergerea după pereche — `DELETE /api/students/{sid}/enrollments/{cid}`. Semnalat la runda 3 (M5).

M7 — PUT /api/books/{id} cere createdAt obligatoriu și îl suprascrie. `BookUpdateRequest.java:12` + `BookCommandServiceImpl.java:78` . Clientul e obligat să trimită data de creare la fiecare update și poate s-o schimbe. Data la care un rând a fost creat nu e un câmp editabil — o pune serverul o dată, la creare. Dacă vrei să rămână în DTO, măcar ignoră-o la scriere.

M8 — ștergerea unui curs șterge în tăcere înscrierile studenților. Reprodus: `DELETE /api/courses/{id}` pe un curs cu 5 înscrieri → 200, cursul și toate cele 5 înscrieri dispar (`cascade = CascadeType.ALL` + `orphanRemoval = true` pe `Course.enrollments`). Poate e ce vrei, dar e o decizie de business pe care acum o ia o adnotare: un click șterge istoricul a cinci studenți, fără avertisment și fără nimic în răspuns care s-o spună. Alternativa uzuală: refuzi ștergerea cât timp există înscrieri (409), sau marchezi cursul ca inactiv.

M9 — niciun GET /{id} , pe nicio resursă. `GET /api/students/{id}` → 405. Poți șterge un student, dar nu-l poți citi. Mai larg: ai 25 de metode scrise în cele patru query service-uri și 4 expuse prin HTTP — 21 nu sunt chemate de nimeni. Fișa separată `ENDPOINTURI-LIPSA.pdf` le ia pe rând.

M10 — structura cerută la runde 1, 2 și 3, neatinsă. `ExceptionConstants` în patru pachete cu același nume `services/interfaces/` în loc de `services/` + `services/impl/` . plural inconsistent: `models` , `dtos` , `services` , `controllers` , dar `repository` la singular.

M11 — trei convenții de nume în același răspuns JSON. `EnrollmentCreateResponse` întoarce `enrollmentId` , `studentId` , `courseId` ; `EnrollmentDeleteResponse` întoarce `enrollmentID` , `studentID` , `courseID` ; `EnrollmentUpdateResponse` întoarce `id` , `studentId` , `courseId` . Trei răspunsuri ale aceluiași modul, trei forme diferite pentru același câmp. M-am împiedicat efectiv de asta când scriam probele: am citit `id` din răspunsul lui POST și am primit gol. Un client care consumă API-ul tău trebuie să ghicească de fiecare dată.

Cleanups

- **C1** — `Enrollment.java:9` : `import org.springframework.cglib.core.Local;` — nefolosit, ales din auto-complete. Semnalat la runde 2 și 3.
- **C2** — `CourseSummary.java:5` : `import java.util.function.Function;` — nefolosit. Acelaș tipar ca C1: importul vechi (`CourseRepository`) a plecat, unul nou a intrat în loc.
- **C3** — `BookController.java:5` : `import ...book.models.Book;` nefolosit. Semnalat (C7).
- **C4** — `EnrollmentCommandServiceImpl.java:29` : constructorul primește `studentRepository` și `courseRepository` și nu le atribuie — acum au rămas și un rând gol la `:31` , unde era atribuirea. Spring le injectează degeaba. Semnalat la runda 3 (C4).
- **C5** — `BookQueryServiceImpl.java:14` e `@Component` ; celelalte trei sunt `@Service` . Semnalat la runde 2 și 3.
- **C6** — `deletebook` , `updatebook` cu literă mică, în interfață și în impl. Semnalat la runde 2 și 3 (C8, C5).
- **C7** — `StudentUpdateRequest.java:19` : `@NotNull` pe un `int` nu se poate declanșa niciodată — un primitiv nu e null. Ce te apără acolo e `@Positive` de dedesubt: JSON fără `age` ajunge 0 și `@Positive` îl respinge. Adnotarea e inofensivă, dar e o regulă care pare să facă ceva și nu face.
- **C8** — `StudentUpdateRequest.java:5` : `import java.util.UUID;` nefolosit.
- **C9** — `Student.java:75-81` : constructorul `Student(String text)` care parsează CSV, rămas din vremea lui `Runner` . Nimic din API nu-l cheamă.

- **C10** — `Runner.java` are peste 700 de linii, din care majoritatea în metode de test comentate. Odată ce ai controllere și probe HTTP, locul lor e la gunoi.

Before / After (doar pentru 🚫)

#	Acum	Corect
B1	<code>@JoinColumn(name="student_id", unique=true, nullable=false)</code> pe amândouă	fără <code>unique=true</code> ; unicitatea perechii o ține <code>@Table(uniqueConstraints=...)</code>
B2	<code>reset(); seed();</code> rulate la fiecare pornire	<code>@Profile("seed")</code> pe <code>Runner</code> , pornit doar când vrei date de test
B3	nicio clasă de advice	<code>@RestControllerAdvice</code> cu <code>@ExceptionHandler</code> pe familie: not-found → 404, conflict → 409, validare → 400, cu <code>message</code> în corp
B4	<code>!findByStudentId(sid).isEmpty() && existsByName(name)</code>	<code>existsByStudentIdAndName(sid, name)</code> — o întrebare, despre pereche
B5	<code>findById(id).orElseThrow()</code>	<code>findById(id).orElseThrow(StudentNotFound::new)</code> — în toate 7 locurile
B6	<code>PUT</code> scrie emailul/numele fără nicio verificare	<code>findByEmail(email).filter(s -> !s.getId().equals(id))</code> → <code>EmailAlreadyUsed</code>
B7	<code>findByStudentIdAndCourseId(...).isPresent()</code>	<code>.filter(e -> !e.getId().equals(id)).ifPresent(...)</code>
B8	<code>if (list.isEmpty()) { throw new NoStudentsFound(); }</code>	<code>return list;</code> — lista goală e <code>200 []</code>

Q&A — de răspuns înainte de runda 5

1. **B1.** Aplicația ta pornește, a mea nu — același commit, același `ddl-auto: update` . Ce anume diferă între cele două baze de date, și de ce constrângerea din `@Table` s-a adăugat pe baza existentă, iar indexul de pe `@JoinColumn` nu? Ce test ai putea rula, înainte de fiecare push, care ar fi prins asta din prima?
2. **B4.** `POST {Effective Java, student B}` e refuzat, deși B nu are cartea aceea. Scrie în cuvinte ce întreabă fiecare dintre cele două condiții din `if` , separat — și spune care e întrebarea pe care voiai de fapt s-o pui. De ce un `&&` între ele nu o compune?

3. **B6 + B7**. Ambele sunt despre unicitate la update, și fixul are aceeași formă în amândouă: „există altul, în afară de mine?”. De ce verificarea copiată din metoda de creare nu funcționează la update — ce s-a schimbat între cele două situații?
-

Runda 3 — 2026-09-23 — sha 62915d4

- ✓ B4 `@Valid` fără reguli pe `EnrollmentUpdate/DeleteRequest` — **închis** (`PUT {}` → 400)
- ✓ B5 `createBook` nu putea reuși niciodată — **închis** (`POST /api/books` → 200)
- ✓ B6 trei command service-uri inaccesibile din API — **închis** (CRUD pe toate patru)
- B1 liste goale → 500 — **NEATINS**, 13 locuri (runda 4, B8)
- B2 niciun `@RestControllerAdvice` — **NEATINS**, acum 12 situații (runda 4, B3)
- B3 `PUT` pe aceeași pereche → 500 — **NEATINS** (runda 4, B7)
- ✓ M6 `Enrollment.equals` crăpa fără student/curs — **închis**
- ✓ M7 `deleteStudent` ignora emailul — **închis** (semnătura trecută pe `UUID`)
- M2 query dublu — **închis** în `CourseQueryService` , rămas în celelalte (runda 4, M3)
- M8 unicitate lipsă la update — **NEATINS**, acum expus prin HTTP (runda 4, B6)
- M1 graf cu două colecții · M3 `== null` imposibil · M4 `POST` → 200 · M5 `DELETE` cu corp · M9 `@Transactional` lipsă · M10 structura — **toate NEATINSE** (runda 4, M1–M10)
- C1, C2, C3, C4, C5, C7 — **neatinse** (runda 4, C1–C6)

Runda 2 — 2026-08-19 — sha 8b84a6d

- ✓ B1 `Course.enrollments` neinițializat → NPE — **închis**
- ✓ B2 `StudentSummary.getId()` `Long` peste `UUID` — **închis**
- ✓ B3 `@EntityGraph` către un graf inexistent — **închis**
- B4 `createBook` nu poate reuși niciodată — **NEATINS** (runda 3, B5)
- B5 `updateEnrollment` duplică perechea — verificarea a fost adăugată, dar prinde și rândul propriu (runda 3, B3); constrângerea `UNIQUE` pe tabelă tot lipsește
- B6 `Enrollment.equals` crapă fără student/curs — **NEATINS** (runda 3, M6)
- B7 `deleteStudent` ignoră emailul din request — **NEATINS** (runda 3, M7)
- M1 `updateCourse` scrie orbește · M2 email duplicat la update · M3 excepție greșită · M4 query dublu · M5 `== null` imposibil · M6 `@Transactional` lipsă · M7 factory · M8 `Enrollment` incomplet — **toate NEATINSE** (runda 3, M2–M9)
- C1–C8 — **neatinse**, cu excepția celor dispărute odată cu refacerea fișierelor

Runda 1 — 2026-08-05

- ✓ `spring-boot-starter-web` + `controllers/` — **închise la runda 3**
- `@RestControllerAdvice` — **NEATINS de trei runde** (runda 3, B2)

-  `services/interfaces/` → `services/` + `services/impl/` — **NEATINS**
-  `ExceptionConstants` duplicat (acum în 4 pachete) — **NEATINS**
-  plural inconsistent `repository` vs `models / dtos / services` — **NEATINS**
- Decizia de arhitectură: `Student` = aggregate root, `book/` fără `services/` proprii — contrazisă de `BookCommandService`, de reluat explicit

